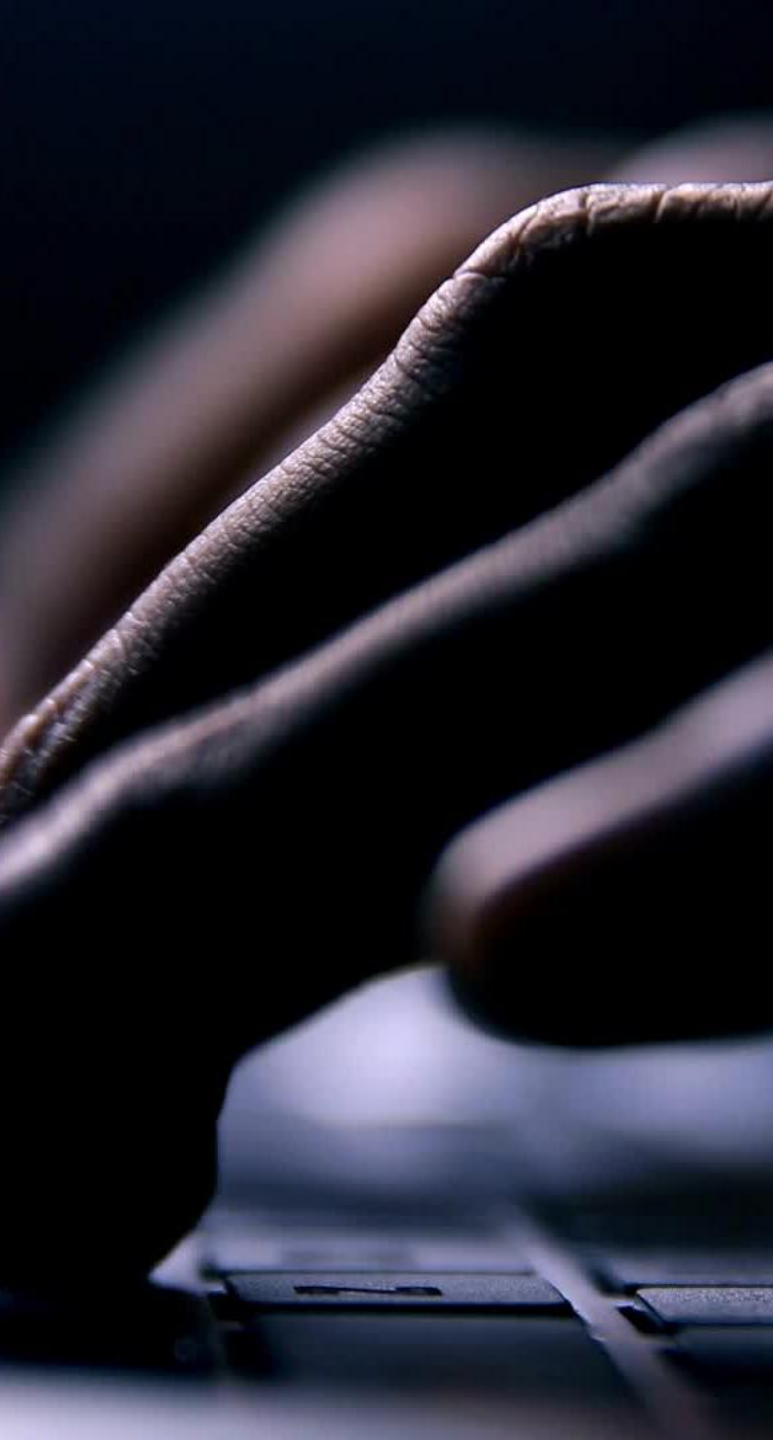


Fatec
Diadema
Luigi Papaiz

Integração e Entrega Contínua

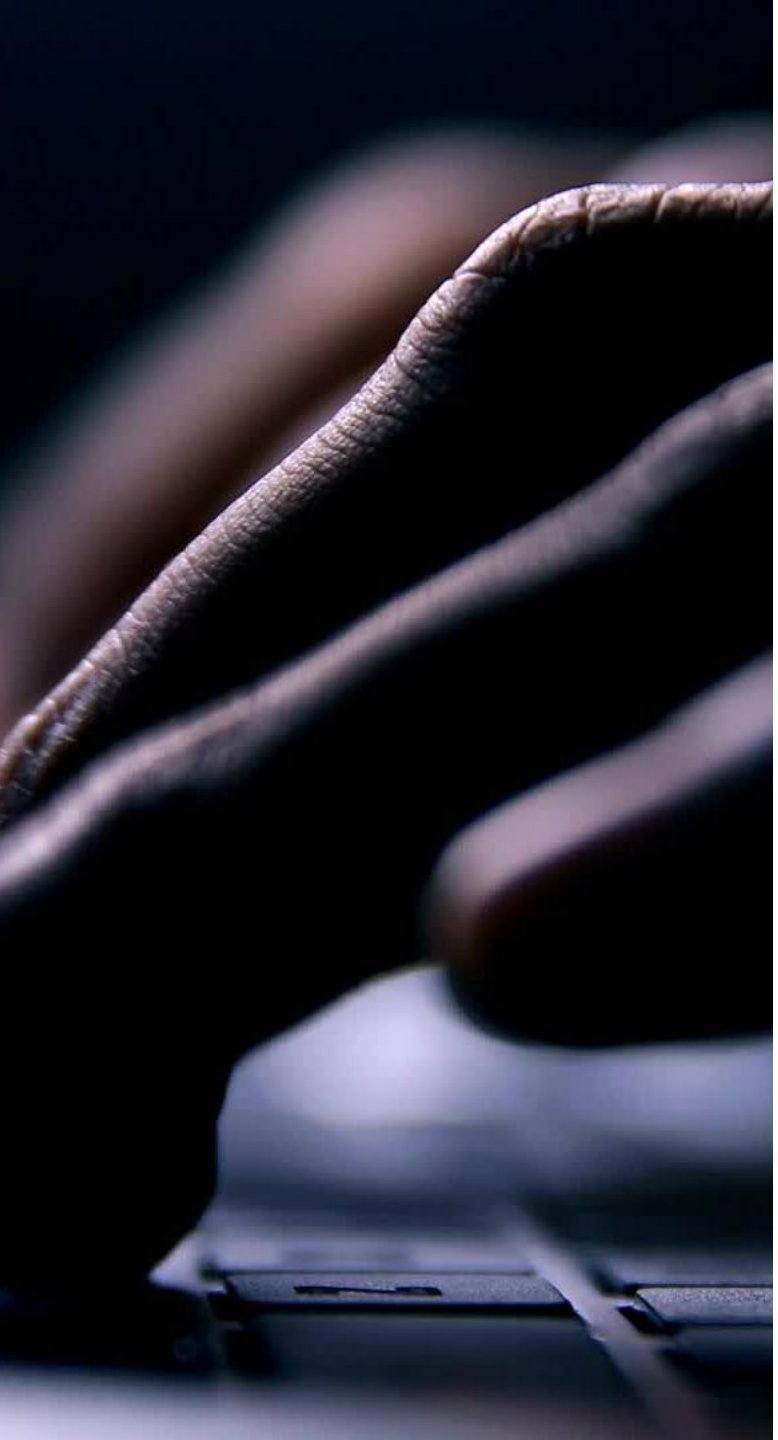
Prof. Rafael Miranda

rafael.miranda@cps.sp.gov.br

Na aula anterior...

Fatec
Diadema
Luigi Papaiz

- Pirâmide de teste
- Linters
- Cobertura de Testes (Code Coverage)



Fatec
Diadema
Luigi Papaiz

Qualidade de Software no Pipeline II

Análise Estática de Código

Fatec
Diadema
Luigi Papaiz

- Alguns problemas não aparecem nos testes.
  - Exemplo:
    - código duplicado
    - funções muito grandes
    - variáveis não usadas
    - problemas de segurança
- Ferramentas de **análise estática** analisam o código sem executá-lo.

Dívida Técnica e Code Smells

Fatec
Diadema
Luigi Papaiz

- **Dívida Técnica:** O custo de escolher uma solução fácil agora em vez de uma abordagem melhor que levaria mais tempo.
- **Code Smells (Mau Cheiro no Código):** Sintomas que indicam um problema profundo (ex: funções gigantescas, muitos parâmetros, código duplicado).
- O CI deve atuar como um **gate**, impedindo que a dívida técnica cresça descontroladamente.

Introdução ao SonarQube / SonarCloud

Fatec
Diadema
Luigi Papaiz

- Uma das ferramentas mais poderosas do mercado para CI/CD.
- O que ele mede:
  - **Bugs:** Erros lógicos.
  - **Vulnerabilidades:** Pontos de invasão (Segurança).
  - **Code Smells:** Problemas de manutenibilidade.
  - **Duplicação:** Código copiado e colado.
- Ele gera um "Quality Gate" (Portão de Qualidade).

Incluir o Sonar Cloud no pipeline

Fatec
Diadema
Luigi Papaiz

- Seguir o passo a passo do arquivo aula 05.1 – Passo a Passo Sonar Cloud

Pipeline Completo de Qualidade

Fatec
Diadema
Luigi Papaiz

- Pipeline real:

Commit
Install dependencies
Lint
Unit Tests
Coverage
Static Analysis (Sonar)
Deploy

- Objetivo:
  - Garantir qualidade antes do deploy.

Revisão de Código (Code Review) vs Automação

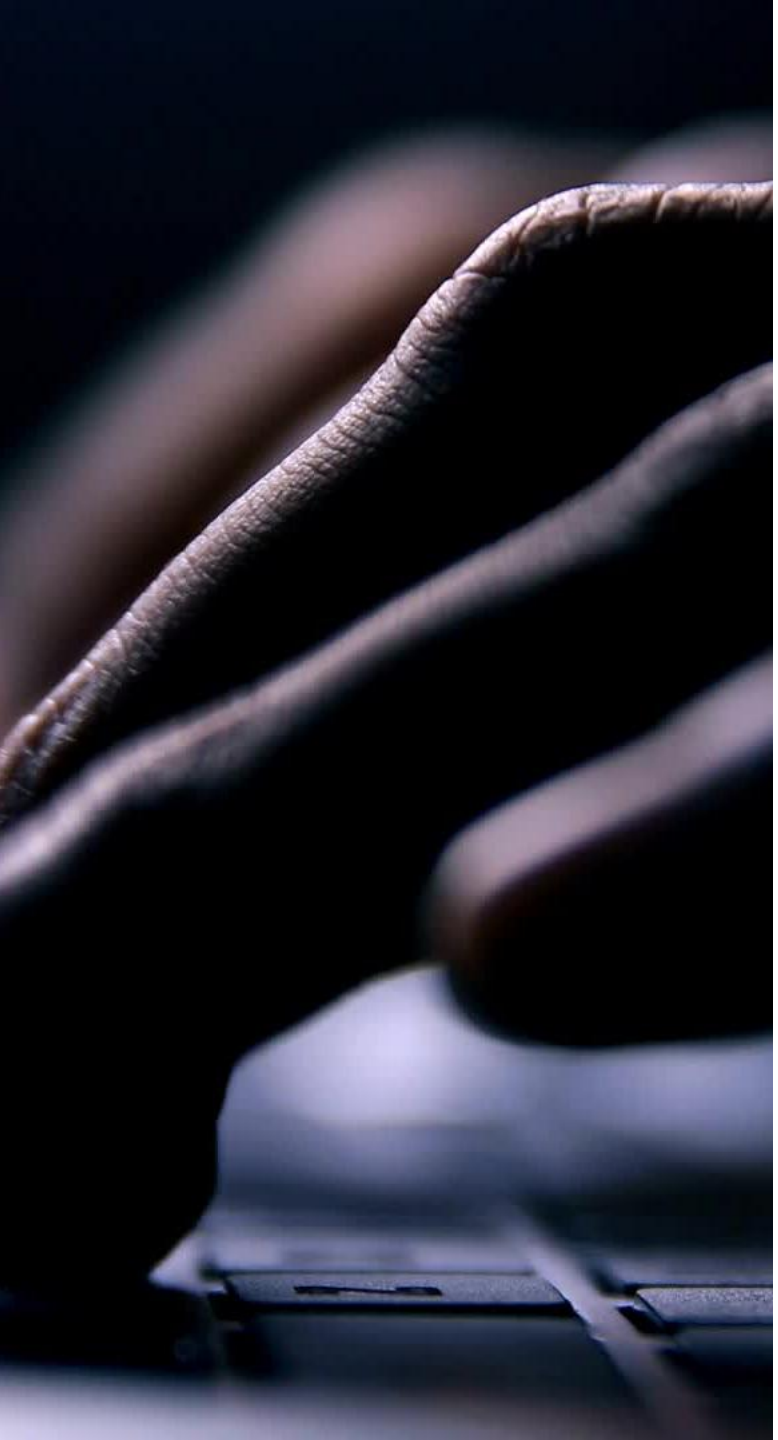
Fatec
Diadema
Luigi Papaiz

- A automação (Linter/Sonar) resolve as discussões bobas (espaços, aspas, nomes).
- Isso libera o ser humano para revisar a **arquitetura e a regra de negócio** no Pull Request.
- Menos tempo discutindo "vírgulas", mais tempo discutindo "soluções".

Desafio

Fatec
Diadema
Luigi Papaiz

- Criar um projeto simples com um **pipeline de CI completo**, aplicando conceitos de qualidade de software.
- Você pode escolher:
  - Linguagem (JavaScript, Python, Java, etc.)
  - Tipo de projeto (API, calculadora, sistema simples, etc.)
- **Requisitos do Projeto**
  - Mínimo de **3 funcionalidades**
  - Mínimo de **3 testes automatizados**
  - Projeto funcional (não precisa ser complexo)



Fatec
Diadema
Luigi Papaiz

Pipeline como escudo de Segurança

Pipeline Atual

Fatec
Diadema
Luigi Papaiz

- Mesmo passando em todas as etapas, o sistema ainda pode estar vulnerável.
- **Exemplos:**
  - Biblioteca com falha conhecida
  - Código exposto a ataques
    - **Insight:** qualidade ≠ segurança

Evolução do pipeline

Fatec
Diadema
Luigi Papaiz

Código → Lint → Testes → Qualidade → Segurança → Build → Deploy

- **Conceito:**
  - Segurança é mais um *gate* dentro do CI/CD.
- **Interpretação:**
  - Teste falha → bug
  - Segurança falha → risco real

O que é DevSecOps?

Fatec
Diadema
Luigi Papaiz

- Integração de práticas de segurança no pipeline de desenvolvimento.
- **Ideia central:**
  - Shift Left Security → trazer segurança para o início do processo.
- **Por quê isso importa?**
  - Falhas em produção são mais caras
  - Impacto direto no usuário
  - Pode gerar vazamento de dados

Tipos de validação no pipeline

Fatec
Diadema
Luigi Papaiz

Tipo	Objetivo
Testes	Detectar bugs
SonarCloud	Melhorar qualidade
Segurança	Evitar vulnerabilidades

Cada etapa resolve um problema diferente

Segredos no código (Secrets)

Fatec
Diadema
Luigi Papaiz

- **Erro comum:**

```
const apiKey = "123456";
```

- **Problema:**
  - Exposição de credenciais
  - Acesso indevido

Boas práticas para secrets

Fatec
Diadema
Luigi Papaiz

- **Soluções:**
  - GitHub Secrets
  - Vault
  - Variáveis de ambiente
- **Regra de ouro:**

  Credenciais nunca devem estar no código

Secret Scanning

Fatec
Diadema
Luigi Papaiz

- Ferramentas que detectam segredos expostos no código.
- **Exemplo:**
  - Token AWS
  - Chave de API
- **No CI/CD:**
  - Pode bloquear o pipeline
  - Pode impedir o commit

Exemplo prático no Pipeline

Fatec
Diadema
Luigi Papaiz

- **Exemplo de código inseguro**
  - const password = "my_super_secret_password";
- **Pipeline com verificação de segredos**

```
- name: Secret Scan

  uses: gitleaks/gitleaks-action@v2
```

SCA (Software Composition Analysis)

Fatec
Diadema
Luigi Papaiz

- Análise de bibliotecas externas usadas no projeto.
- **Fato importante:**
  - Grande parte do código vem de terceiros.
- **Risco:**
  - Se uma dependência tiver vulnerabilidade conhecida, aumenta o risco de ataque.

Exemplo real

Fatec
Diadema
Luigi Papaiz

- Dependência vulnerável:

  lodash 4.17.20

- Vulnerabilidade:

  Prototype Pollution

  Severity: HIGH

Verificando vulnerabilidades

Fatec
Diadema
Luigi Papaiz

npm audit

- **O que retorna:**
  - Lista de vulnerabilidades
  - Nível de severidade
- **Classificação:**
  - Low
  - Moderate
  - High
  - Critical

Segurança no pipeline

Fatec
Diadema
Luigi Papaiz

- Rodar no Terminal:

  npm install lodash@4.17.20

- Exemplo no GitHub Actions:

  - name: Verificar vulnerabilidades

  run: npm audit --audit-level=high

- **Comportamento:**
  - Se houver vulnerabilidade HIGH → pipeline falha
  - Segurança vira critério de aprovação

SAST (Static Application Security Testing)

Fatec
Diadema
Luigi Papaiz

- Análise estática do código para identificar padrões inseguros.
- **O que detecta:**
  - SQL Injection
  - Uso de funções perigosas
  - Falta de validação
- **Ferramentas comuns**
  - SonarCloud
  - Semgrep
  - GitHub CodeQL

Exemplo real de código vulnerável

Fatec
Diadema
Luigi Papaiz

```
function executarCodigo(codigo) {
  eval(codigo);
}
```

- Problema:
  - eval() executa qualquer código recebido → risco de **Remote Code Execution**.

Dependabot

Fatec
Diadema
Luigi Papaiz

- Ferramenta automática de gerenciamento de dependências.
- **O que faz:**
  - Detecta vulnerabilidades
  - Sugere atualização
  - Cria Pull Request
- **Benefício:**

  Mantém o projeto seguro continuamente